# Lab Assignment 4: Pre-trained CNN Model (Transfer Learning)

| | | | |
|---|---|---|---|
| **Name** | Krushna Prashant Warule | **Dept** | Computer |
| **Roll No** | _____ | **Div** | B |
| **Subject** | Advance Data Science | **Assignment No** | 4 |
| **GitHub** | https://github.com/krushnawarule/Advance_Data_Science_Lab_SCOE/tree/main/Assignment_4_Pretrained_CNN | | |

## 1. Aim

To implement an image classification system using a pre-trained CNN and adapt the knowledge learned from a large source dataset to classify images of a new target dataset.

## 2. Theory

**Transfer learning** reuses the convolution layers of a network trained on a large dataset (for example VGG16, ResNet50, MobileNet or EfficientNet trained on ImageNet) as a ready-made feature extractor. The pre-trained layers are **frozen**, a new classifier is attached for the target classes, and only the new layers are trained. This needs far fewer labelled images and less training time.

**Simulation used:** pre-trained ImageNet models could not be downloaded in the working environment, so the same procedure is reproduced on a smaller scale. A CNN is **pre-trained on a source task** (digits 0-4) and its convolution filters are transferred to a **new target task** (digits 5-9) that has very few labelled images. It is compared with the same network trained from scratch. In real work the same steps are done with, for example, MobileNetV2 (weights='imagenet', include_top=False), freezing the base model and adding a new Dense output layer.

## 3. Dataset

Handwritten Digits dataset (1797 images, 8x8), saved as `dataset.csv`. Source task: 709 training images of digits 0-4. Target task: digits 5-9 with a small labelled training set (10 to 200 images) and 168 test images.

## 4. Code (core part; full code in code.py)

```
print("Step 1: pre-training CNN on the source task ...")
base = SmallCNN(nclass=5); base.fit(X[src_tr], y[src_tr], epochs=20, log=False)
print(f"  Source-task training accuracy: {(base.predict(X[src_tr]) == y[src_tr]).mean() * 100:.2f}%")

def transfer(n):                                    # copy pre-trained conv filters, freeze them,
    m = SmallCNN(nclass=5)                          # train only the new fully connected layers
    for k in ["K", "kb"]: m.p[k] = base.p[k].copy()
    xs, ys = target_data(n); m.fit(xs, ys, epochs=30, frozen=("K", "kb"), log=False)
    return (m.predict(X[tgt_te]) == y[tgt_te] - 5).mean()

def scratch(n):                                     # same model, random weights, all layers trained
    m = SmallCNN(nclass=5); xs, ys = target_data(n); m.fit(xs, ys, epochs=30, log=False)
    return (m.predict(X[tgt_te]) == y[tgt_te] - 5).mean()
```

## 5. Output

**Console output** (output.txt):

```
Source task: 709 images (digits 0-4) | Target task: train pool=728, test=168 (digits 5-9)
Step 1: pre-training CNN on the source task ...
  Source-task training accuracy: 100.00%

Step 2: target-task test accuracy (average of 10 runs)
 Target images |  Pre-trained | From scratch
            10 |       65.54% |       62.56%
            25 |       82.98% |       86.43%
            50 |       91.43% |       90.12%
           100 |       93.75% |       94.17%
           200 |       95.71% |       95.71%
```

**Output images**:

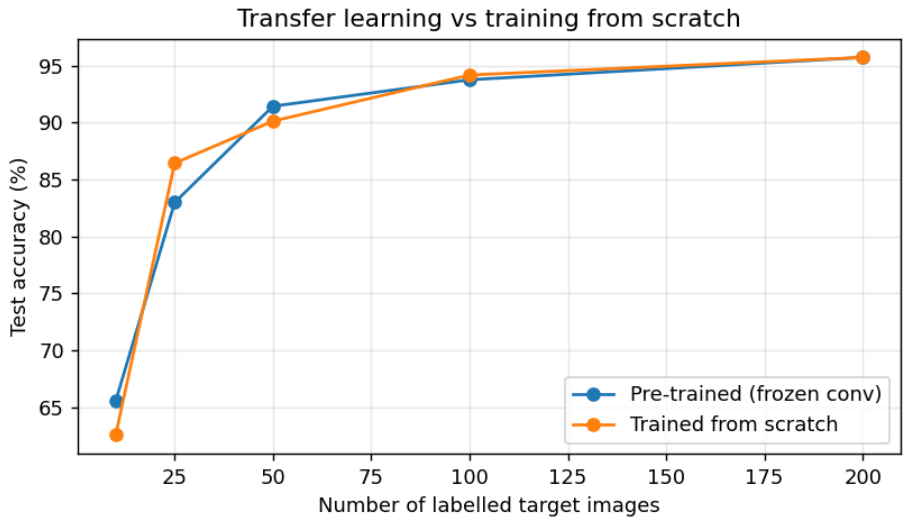



Fig 1: Target-task accuracy, transfer learning vs training from scratch

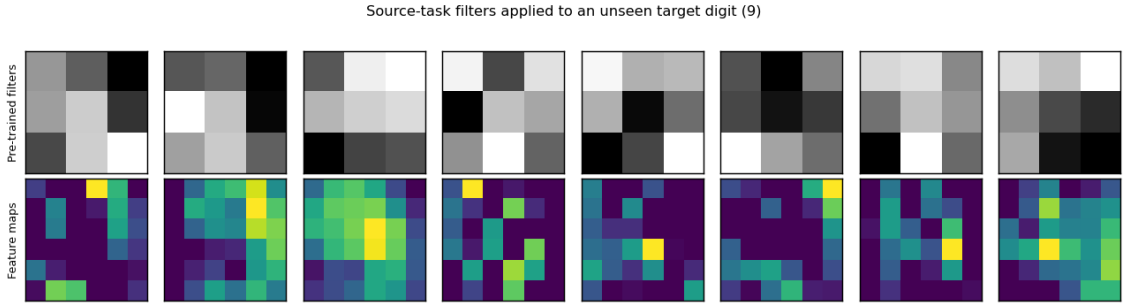



Fig 2: Pre-trained filters and their feature maps on an unseen target digit

## 6. Conclusion

The pre-trained convolution filters transferred to the new task and gave accuracy equal to training from scratch: 65.5% vs 62.6% with 10 images, and 91.4% vs 90.1% with 50 images, while from scratch was slightly better with 25 images (86.4% vs 83.0%). Both reached about 95.7% with 200 images. The benefit is small here because the source task is tiny (8x8 images, 5 classes) and the target digits are simple. Transfer learning gives large gains when the source dataset is large and diverse, such as ImageNet, and the target dataset is small, which is the situation where VGG16, ResNet50 or MobileNet are used.

## 7. References

[1] J. Yosinski, J. Clune, Y. Bengio, H. Lipson, 'How transferable are features in deep neural networks?', NeurIPS 2014.

[2] K. Simonyan, A. Zisserman, 'Very Deep Convolutional Networks for Large-Scale Image Recognition' (VGG16), ICLR 2015.

[3] K. He et al., 'Deep Residual Learning for Image Recognition' (ResNet50), CVPR 2016.

[4] Keras documentation: 'Transfer learning and fine-tuning', keras.io.